

Advanced Machine Learning

DATA 642

Lecture 14 — Introduction to Autoencoders

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Representation Learning and Generative Learning Using Autoencoders

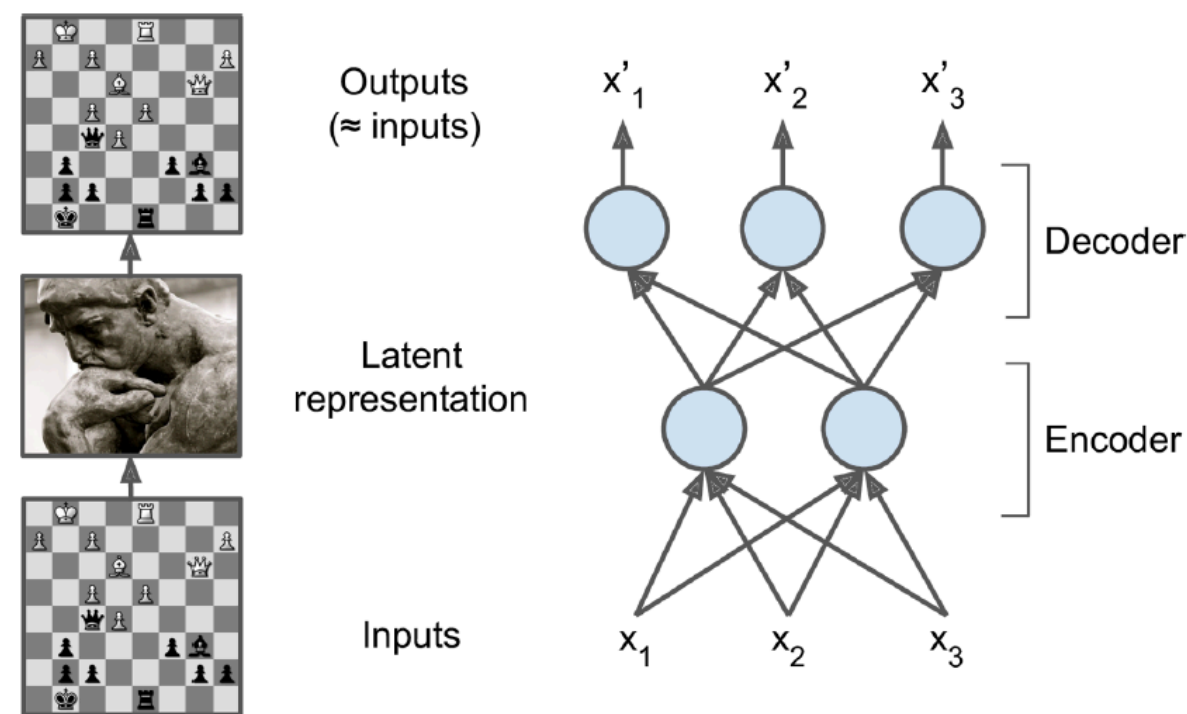
- Autoencoders are artificial neural networks capable of learning dense representations of the input data, called latent representations or codings, without any supervision (i.e., the training set is unlabeled).
- These codings typically have a much lower dimensionality than the input data, making autoencoders useful for dimensionality reduction.
- Autoencoders also act as feature detectors, and they can be used for unsupervised pre-training of deep neural networks.
- Some autoencoders are generative models: they are capable of randomly generating new data that looks very similar to the training data. For example, you could train an autoencoder on pictures of faces, and it would then be able to generate new faces.
- Autoencoders learn to replicate their inputs. Limiting latent representation size or introducing noise prevents direct copying, forcing the model to learn efficient data representations. Thus, the codings emerge as byproducts of the autoencoder's learning process, aiming to approximate the identity function within set constraints.

Applications of Autoencoders

- **Dimensionality Reduction:** Autoencoders are used to reduce the dimensionality of data while retaining essential information. This is beneficial for tasks like data visualization and preprocessing.
- **Anomaly Detection:** Autoencoders can reconstruct normal data accurately. Anomalies often result in poor reconstructions, making autoencoders useful for anomaly detection in various domains, such as fraud detection and cybersecurity.
- **Feature Learning:** Autoencoders learn compact representations of input data, capturing important features. These learned representations can be used as feature vectors for downstream tasks like classification and regression.
- **Data Denoising:** By training on noisy data and reconstructing clean data, autoencoders can effectively denoise data. This is useful in various applications such as image denoising and signal processing.
- **Generative Modeling:** Variational autoencoders (VAEs) and generative adversarial networks (GANs) use autoencoder architectures to generate new data samples. This has applications in image generation, text-to-image synthesis, and data augmentation.

The technicalities

- An autoencoder transforms inputs into a concise latent representation and generates outputs resembling the inputs. It consists of two components: an encoder for conversion to latent representation and a decoder for reconstructing the inputs.



- An autoencoder shares architecture with Multi-Layer networks but requires the output layer to match the input size. Outputs, termed reconstructions, are aimed to mirror inputs. A reconstruction loss in the cost function penalizes deviations between reconstructions and inputs.

In autoencoders, the choice of loss function depends on the nature of the problem and the type of data being reconstructed. Here are some common loss functions used in autoencoders:

Mean Squared Error (MSE) Loss:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2, \quad (1)$$

where $\hat{x}_i$ is the output of the autoencoder (reconstruction).

Binary Cross-Entropy Loss (for binary data):

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N (x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i)) \quad (2)$$

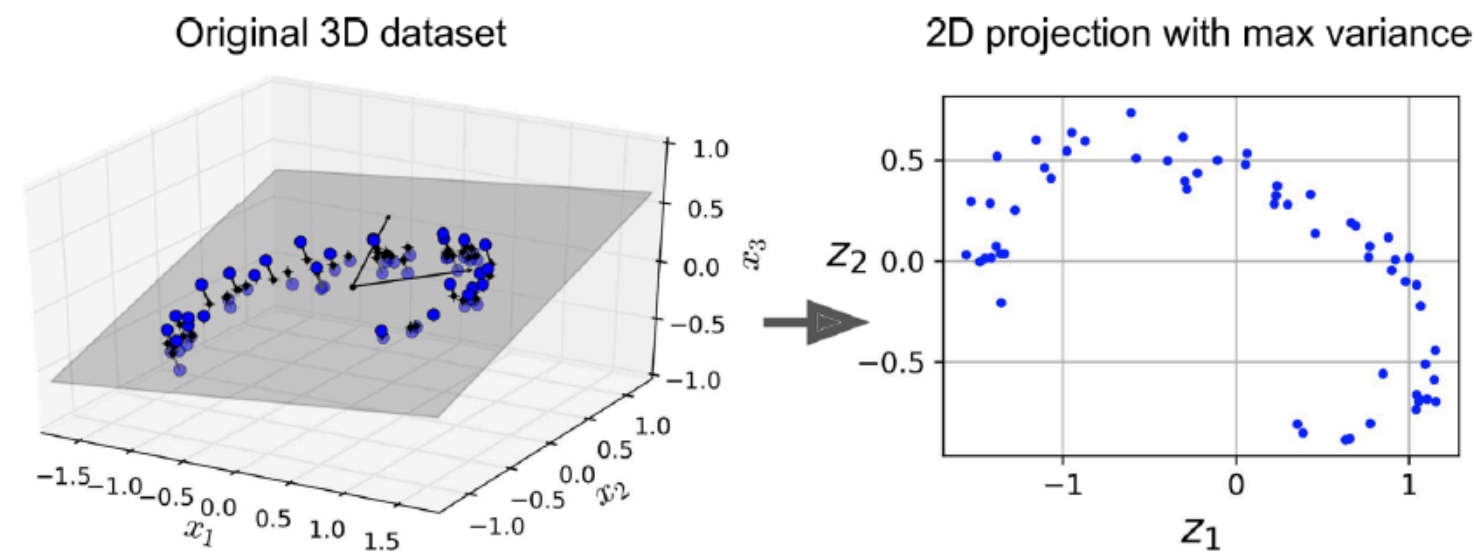
Categorical Cross-Entropy Loss (for categorical data):

$$\text{CCE} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C x_{ij} \log(\hat{x}_{ij}), \quad (3)$$

where x_{ij} is the one-hot encoded input data for category j (1 if category j is active, 0 otherwise), $\hat{x}_{ij}$ is the output of the autoencoder (reconstruction) for category j and C is the number of categories.

Dimensionality reduction with a linear Autoencoder

- Linear autoencoders, like PCA, reduce input dimensionality while preserving essential information by identifying principal components or latent representations capturing maximal data variance. PCA finds orthogonal principal components, while a linear autoencoder compresses input data into a lower-dimensional space without activation functions and reconstructs it.
- Both techniques aim for dimensionality reduction while retaining crucial data information.
- Both PCA and linear autoencoders are unsupervised learning methods. Linear autoencoders, being linear, resemble PCA closely, although they can handle non-linear relationships with activation functions.



```
np.random.seed(42)
tf.random.set_seed(42)

encoder = keras.models.Sequential([keras.layers.Dense(2, input_shape=[3])])
decoder = keras.models.Sequential([keras.layers.Dense(3, input_shape=[2])])
autoencoder = keras.models.Sequential([encoder, decoder])

autoencoder.compile(loss="mse", optimizer=keras.optimizers.SGD())
```

Stacked Autoencoders

- Just like other neural networks we have discussed, autoencoders can have multiple hidden layers. In this case they are called **stacked autoencoders** (or **deep autoencoders**).
- Adding more layers helps the autoencoder learn more complex codings.
- The architecture of a stacked autoencoder is typically symmetrical with regard to the central hidden layer (the coding layer).
- Stacked autoencoders are beneficial for learning hierarchical representations of data, enabling them to capture increasingly abstract features in deeper layers.

